Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

Chapter 1
Introduction

This book presents the fundamentals of sparse matrix algorithms for the direct solution of sparse linear systems, from theory to algorithms and data structures to working code. The algorithms presented here have been chosen with these goals in mind: they must embody much of the theory behind sparse matrix algorithms; they must be either asymptotically optimal in their run time and memory usage or be fast in practice; they must be concise so as to be easy to understand and short enough to print in their entirety in this book; they must cover a wide spectrum of matrix operations; and they must be accurate and robust.

Algorithms are presented in a mixture of pseudocode, MATLAB, and C, so knowledge of these is assumed. Also required is a basic knowledge of linear algebra, graph theory, algorithms, and data structures. This background is reviewed below and in an appendix on the C programming language.

Chapter 2 presents basic data structures and algorithms, including matrix multiplication, addition, transpose, and data structure manipulations. Chapter 3 considers the solution of triangular systems of equations. Chapters 4 through 6 present the three most commonly used decompositions: Cholesky, QR, and LU. Factorization methods specifically for symmetric indefinite matrices are not discussed. Section 4.10 presents a method for updating and downdating a sparse Cholesky factorization after a low-rank change. Chapter 7 discusses ordering methods that reduce work and memory requirements. Chapter 8 draws on the theory and algorithms presented in Chapters 1 through 7 to solve a sparse linear system $Ax = b$, where A can be symmetric positive definite, unsymmetric, or rectangular, just like the backslash operator in MATLAB, `x=A\b`, when `A` is sparse and `b` is a dense column vector. Chapter 9 is a summary of the CSparse sparse matrix package. Finally, Chapter 10 explains how to use sparse matrices in MATLAB.

To avoid breaking the flow of discussion, few citations appear in the body of each chapter. They are discussed at the end of each chapter instead in a "Further reading" section, which gives an overview of software, books, and papers related to that chapter. Notable exceptions to this rule are the theorems stated in the book. The final section in each chapter is a set of exercises for that chapter.

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

1.1 Linear algebra

Definitions and notation for linear algebra are briefly described below. An m-by-n *matrix* is denoted with a capital letter, $A \in \mathbb{R}^{m\times n}$, where $\mathbb{R}^{m\times n}$ denotes the set of m-by-n matrices with real elements. An ij subscript on the corresponding lower case letter a_{ij} denotes an entry in row i and column j of the matrix A. It can also denote a row vector, column vector, or block submatrix of A (described below), but in this case i and j are typically small constants (a_{12}, for example). L is a lower triangular matrix, U and R are upper triangular, and I denotes the *identity matrix* (which is zero except for $(I)_{ii} = 1$). A matrix A is *lower triangular* if $a_{ij} = 0$ when $i < j$, and *upper triangular* if $a_{ij} = 0$ when $i > j$. Lower case letters denote vectors (b, x, y, z), except for letters i through n, which always denote integer scalars. Row vectors are shown with a superscript, x^T. With a subscript, x_i can denote either a scalar or a vector, depending on the context. Lower case Greek letters (α, β) represent scalars. The jth column of the matrix A is denoted A_{*j}, or `A(:,j)` in MATLAB. Likewise, A_{i*} denotes row i of A, or `A(i,:)` in MATLAB.

The *transpose* A^T of the real matrix A is defined by $(A^T)_{ij} = a_{ji}$. The matrix *addition* $C = A + B$ is defined by $c_{ij} = a_{ij} + b_{ij}$. A scalar can be multiplied by a matrix; $C = \alpha A$ is defined by $c_{ij} = \alpha a_{ij}$. Matrix *multiplication*, $C = AB$, is

$$c_{ij} = \sum_{k=1}^{s} a_{ik} b_{kj}, \tag{1.1}$$

where $C \in \mathbb{R}^{m\times n}$, $A \in \mathbb{R}^{m\times s}$, and $B \in \mathbb{R}^{s\times n}$. Matrix multiplication requires the number of columns of A to be equal to the number of rows of B. The *dot product* of two vectors x and y is the scalar $x^T y$. The *outer product* of two vectors x and y is the matrix xy^T; the computation of $A = A + \alpha xy^T$ is called a rank-1 *update*. If X and Y are matrices with k columns, XY^T is referred to as a *rank-k outer product* (this nomenclature is commonly used even if X and Y do not have a numerical rank of k). The computation $A = A + \alpha XY^T$ is called a rank-k update.

A *block matrix* is a matrix where each entry can be a matrix, vector, or scalar. Suppose the rows of an m-by-n matrix A are partitioned into r subsets, and the columns are partitioned into c subsets; A can be written as a block r-by-c matrix

$$A = \begin{bmatrix} A_{11} & \cdots & A_{1c} \\ \vdots & & \vdots \\ A_{r1} & \cdots & A_{rc} \end{bmatrix},$$

where A_{ij} is m_i-by-n_j, if m_i is the size of the ith row subset and n_j is the size of the jth column subset. Two block matrices can be added if they are partitioned identically. Two block matrices can be multiplied, $C = AB$, if the columns of A are partitioned identically to the rows of B; the rows of C and A are partitioned identically, as are the columns of C and B. If c is the number of partitions of the columns of A and rows of B, (1.1) becomes

$$C_{ij} = \sum_{k=1}^{c} A_{ik} B_{kj}.$$

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

A set of vectors $a_1, a_2, \ldots, a_n$ is *linearly independent* if $\sum \alpha_j a_j = 0$ implies α_j is zero for all j. The *span* of a set of vectors is the set of vectors that can be written as a linear combination of vectors in the set; $\text{span}(a_1, a_2, \ldots, a_n) = \{\sum \alpha_j a_j\}$. The *range* of a matrix A is the span of its column vectors. The *rank* of a matrix A is the maximal size of the subsets of the columns of A that are linearly independent. An n-by-n matrix is *singular* if its rank is less than n. An m-by-n matrix is *rank deficient* if its rank is less than $\min(m, n)$; it has *full rank* otherwise.

The 1-*norm* of a column vector x or row vector x^T is $\|x\|_1 = \sum |x_i|$, its 2-norm is $\|x\|_2 = \sqrt{\sum x_i^2}$, and its ∞-norm is $\|x\|_\infty = \max |x_i|$. The 1-norm of a matrix is the largest 1-norm of its column vectors. The ∞-norm of a matrix is the largest 1-norm of its row vectors.

The *inverse* of a matrix A is A^{-1}, where $AA^{-1} = A^{-1}A = I$. It exists only if A is square and nonsingular. Two vectors x and y are *orthogonal* if $x^T y = 0$. A matrix Q is *orthonormal* if $Q^T Q = I$. A real square orthonormal Q matrix is called orthogonal, in which case $Q^T Q = QQ^T = I$ (that is, $Q^T = Q^{-1}$ if Q is orthogonal). The 2-norm of a vector x and the product Qx are identical if Q is orthogonal.

The kth *diagonal* of an m-by-n matrix A is a vector d consisting of the set of entries $\{a_{ij}\}$, where $j - i = k$. The term *diagonal*, by itself, refers to the 0th diagonal, or main diagonal, of a matrix. The kth diagonal entry of A is a_{kk}.

The number of nonzero entries (*nonzeros* for short) in a matrix or vector is $|A|$, and $|a|$ denotes the absolute value of a scalar.

A *permutation matrix* P is a row or column permutation of the identity matrix. Any given row or column of P contains a single nonzero entry, equal to 1. The LU *factorization* of a square nonsingular matrix A has the form $LU = A$, where L is lower triangular and U is upper triangular. With *partial pivoting* and row interchanges, the factorization is $LU = PA$. A matrix A is *positive definite* if and only if $x^T Ax > 0$ for all nonzero vectors x. It is *positive semidefinite* if $x^T Ax \geq 0$. The *Cholesky factorization* of a square symmetric positive definite matrix A has the form $LL^T = A$, where L is lower triangular with positive diagonal entries. Pivoting is not required for stability. A square matrix A is *diagonally dominant* by rows if $|a_{ii}| \geq \sum_{j \neq i} |a_{ij}|$ for all i. It is strictly diagonally dominant by rows if $|a_{ii}| > \sum_{j \neq i} |a_{ij}|$ for all i. It is (strictly) diagonally dominant by columns if A^T is (strictly) diagonally dominant by rows. A square strictly diagonally dominant matrix is nonsingular. *Gaussian elimination* without pivoting (a form of LU factorization) is stable for any diagonally dominant matrix (by rows or by columns).

A QR *factorization* of a rectangular matrix A is $QR = A$, where Q is orthogonal and R is upper triangular. For a square matrix A, $Ax = \lambda x$ holds for an *eigenvalue* λ and its *eigenvector* x.

Sets are denoted in calligraphic letters $\mathcal{A}$, $\mathcal{B}$, $\mathcal{C}$, $\mathcal{L}$, $\mathcal{R}$, $\mathcal{V}$, $\mathcal{W}$, $\mathcal{X}$, and $\mathcal{Y}$. These typically arise from the nonzero pattern of the corresponding matrix or vector. For example, $\mathcal{A}_{*j} = \{i \,|\, a_{ij} \neq 0\}$, and $\mathcal{X} = \{i \,|\, x_i \neq 0\}$. The $*$ in the subscript is dropped when the context is clear.

The terms *dense* and *sparse* refer to the data structure used to store a matrix. A matrix $A \in \mathbb{R}^{m \times n}$ is dense if it is stored as a full array of m rows and n columns with mn entries. This is called a `full` matrix in MATLAB. All entries are stored,

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

even if some of them are zero. A sparse matrix is stored in a data structure that can exploit sparsity by not storing numerically zero entries. Numerically zero entries may be stored in a sparse matrix, typically as a result of numerical cancellation.

1.2 Graph theory, algorithms, and data structures

A *graph* $G = (V, E)$ is a set of *nodes* $V = \{1, \ldots, n\}$ and a set of *edges* $E = \{(i, j) \,|\, i, j \in V\}$ connecting those nodes. In an *undirected* graph, (i, j) and (j, i) are the same edge; they are different edges in a *directed* graph. In a directed graph, there can be an edge (i, j) but no edge (j, i). The *neighbors* of a node i, or equivalently the *adjacency set* of i, is $\mathrm{Adj}(i) = \{j \,|\, (i, j) \in E\}$. For a directed graph, this is the *out-adjacency* of node i. The *in-adjacency* of node i is $\mathrm{InAdj}(i) = \{j \,|\, (j, i) \in E\}$. The out-adjacency and in-adjacency of a node in an undirected graph are identical. The *adjacency matrix* A of a graph G is a binary n-by-n matrix with $a_{ij} = 1$ if $(i, j) \in E$, and $a_{ij} = 0$ otherwise. It is symmetric if G is undirected, and may be unsymmetric otherwise. A graph can also be represented as a set of n *adjacency lists*, $\mathcal{A}_{i*} = \{j \,|\, a_{ij} \neq 0\}$ or $\mathcal{A}_{*j} = \{i \,|\, a_{ij} \neq 0\}$. This relates the sparsity pattern of an n-by-n matrix A to its corresponding undirected or directed graph G_A. The diagonal a_{kk} is the self-edge (k, k); it is typically excluded from G_A. The *degree* of a node in an undirected graph is the size of its adjacency list, $|\mathcal{A}_{*i}| = |\mathcal{A}_{i*}| = |\mathrm{Adj}(i)|$. Two graphs are *isomorphic* if they can be made identical by renumbering their nodes.

A node i is *incident* on the edge (a, b) if $a = i$ or $b = i$; the edge (a, b) is also said to be incident on node i. A *node-induced subgraph* $\overline{G} = (\overline{V}, \overline{E})$ of G is defined by a subset of the nodes $\overline{V} \subseteq V$, where $\overline{E} = \{(i, j) \,|\, (i, j) \in E \wedge i \in \overline{V} \wedge j \in \overline{V}\}$. An *edge-induced subgraph* $\overline{G} = (\overline{V}, \overline{E})$ is defined by a set of edges $\overline{E} \subseteq E$, where the nodes $\overline{V}$ are all the nodes incident on any edge in $\overline{E}$.

A graph is completely connected if $E = V \times V$; it has an edge between every pair of nodes. A *clique* is a completely connected subgraph.

A *path* $v_0 \rightsquigarrow v_k$ of length k is a sequence of nodes $(v_0, \ldots, v_k)$ where an edge (v_{i-1}, v_i) exists between each pair of adjacent nodes in the sequence. The path is simple if no node appears more than once. Node j is *reachable* from node i if a path $i \rightsquigarrow j$ exists in the graph. The set of all nodes reachable from i in the graph G is $\mathrm{Reach}_G(i)$. A graph is *strongly connected* if there exists a path from any node to any other node. That is, $\mathrm{Reach}_G(i)$ is the entire graph for any node i. A graph is *connected* if its underlying undirected graph is strongly connected. The *underlying* undirected graph of G is the same as G but with all the edge directions ignored.

A *cycle* is a path $i \rightsquigarrow i$ where the first and last nodes are the same. It is a simple cycle if no edge or node appears more than once, except for i itself. A graph with no cycles is *acyclic*. A directed acyclic graph is often called a *DAG*, for short. An undirected acyclic graph is a *forest*. A *tree* is a connected forest. There is a unique simple path between every pair of nodes in a tree. One node of a rooted tree is designated as the *root* r. The unique node following i in the path $i \rightsquigarrow r$ is called the *parent* p of i; the root itself has no parent. Node i is a *child* of p; a node has at most one parent but can have more than one child. A node with no children is a *leaf*. The path $i \rightsquigarrow r$ is the set of *ancestors* of i. Node i is a *descendant* of all nodes

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

in the path $i \rightsquigarrow r$. The set of *proper* ancestors or descendants of node i excludes node i itself. The *subtree* rooted at node i consists of the subgraph induced by the nodes i and its descendants. In a *postordered tree*, the d proper descendants of any node k are nodes $k-d$ through $k-1$.

A *bipartite* graph is an undirected graph whose nodes are partitioned into two subsets, and every edge connects two nodes in different subsets. A *node separator* is a subset $S \subseteq V$ of the nodes of $G = (V, E)$ such that the graph induced by the nodes $V \setminus S$ is unconnected. An *edge separator* is a subset $S \subseteq E$ of the edges of $G = (V, E)$ such that the graph $G = (V, E \setminus S)$ is unconnected. A *node cover* is a set of nodes $S \subseteq V$ such that all edges E are incident on at least one node in S.

Informally, *asymptotic notation* describes the gist of a function $f(x)$: how fast it grows as a function of x. Scalar factors and lower order terms are ignored. For example, all quadratic polynomials $ax^2 + bx + 1$ are $O(x^2)$, unless $a = 0$. More formally, a function $f(x)$ is $O(g(x))$ if there exist positive constants c and x_0 such that $0 \leq f(x) \leq cg(x)$ for all $x \geq x_0$. This provides an asymptotic upper bound. The asymptotic lower bound is defined similarly. A function $f(x)$ is $\Omega(g(x))$ if there exist positive constants c and x_0 such that $0 \leq cg(x) \leq f(x)$ for all $x \geq x_0$. If $f(x)$ is both $O(g(x))$ and $\Omega(g(x))$, then it has a tight asymptotic bound, $\Theta(g(x))$.

Algorithm analysis refers to methods for determining asymptotic bounds on the run time, memory usage, and other aspects of an algorithm. The run time and other statistics are typically expressed as functions of the size of the input. Worst-case analysis finds an upper bound ($O(...)$) that always holds, regardless of the input. Average-case analysis looks at the typical case. These are often the same but not always. The time complexity of an algorithm is the asymptotic bound on its run time.

An *amortized analysis* considers the total run time of a sequence of related operations. If the time for n operations is $T(n)$, the amortized time for one operation is $T(n)/n$. For example, if a function is called n times and takes one unit of time $n-1$ times but $n+1$ units of time just once, the worst-case time complexity is $O(n)$ for any one usage of this function. However, if the cost of all n operations are added together and amortized across all operations, the amortized cost per operation becomes just two, which is $O(1)$. The latter is a more useful picture of the time complexity of one operation in a sequence.

An example of where amortized time complexity is useful is the *dynamic table* algorithm. Consider a table of size k that starts out with $k = 1$ and an operation that inserts an item at the end of the table. If the size k of the table is insufficient, it is doubled in size, taking $O(k)$ time to make the larger copy. With n insertions, it may appear that total time would be $O(n^2)$, but this is not the case. Even with this extra work of copying the table, the amortized cost of a single insertion is $O(1)$, because only the ith insertions when i is a power of two plus one require a complete copy of the table. For n insertions, the time is

$$n + \sum_{i=0}^{\lfloor \log_2(n-1) \rfloor} 2^i \leq 3n,$$

where $\lfloor x \rfloor$ is the largest integer not greater than x. The total time for n insertions

Downloaded 11/21/13 to 134.99.128.41. Redistribution subject to SIAM license or copyright; see http://www.siam.org/journals/ojsa.php

is less than or equal to $3n$. The amortized time for any one insertion is at most 3.

A common class of graph algorithms consists of methods for traversing the nodes and edges of a graph. The *depth-first search* of a graph starts at a node j and finds all nodes reachable from node j. It explores recursively by always examining the outgoing edges of the latest node i just seen. When all edges of i have been explored, it backtracks to the node from which i was first discovered. Nodes are marked so that they are not searched twice. The time taken by a depth-first search is $O(s + e)$, where $s = |\text{Reach}(i)|$ and e is the number of edges in the subgraph induced by s. This subgraph is connected by the way it is constructed. Traversing the entire graph in a depth-first manner requires the traversal to be repeated until all nodes are visited. A depth-first search produces a list of nodes of a DAG in *topological order*; i appears before j if $i \rightsquigarrow j$ is a path in G.

The *breadth-first search* traverses a graph in a different order. Starting at node i, it first examines all nodes adjacent to i. Next, it examines all nodes j whose shortest path $i \rightsquigarrow j$ is of length 2, then length 3, and so on. Like the depth-first search, it too traverses all nodes in $\text{Reach}_G(i)$. Unlike the depth-first search, it traverses these nodes in order of the shortest path from i, not in topological order.

A graph is denoted as G or $\mathcal{G}$, and $\mathcal{T}$ denotes a tree or forest. $\mathcal{E}$ denotes the element lists in the minimum degree ordering algorithm, discussed in Chapter 7.

1.3 Further reading

Golub and Van Loan [114] provide an in-depth coverage of numerical linear algebra and matrix computations for dense and structured matrices; Strang [193] gives an introduction to linear algebra. Moler [157] provides an introduction to numerical computing with a strong MATLAB focus. Stewart presents an in-depth look at matrix decompositions [190] and eigenvalue problems [191]. Higham [135] discusses the behavior of numerical algorithms in finite precision arithmetic.

Cormen, Leiserson, and Rivest [23] discuss algorithms and data structures and their analysis, including graph algorithms. Kernighan and Ritchie [141] give a concise coverage of the C programming language. Higham and Higham [133], Davis and Sigmon [38], or the online documentation for MATLAB are good places to learn more about MATLAB.

The books by Duff, Erisman, and Reid [53] and George and Liu [89] both deal with direct methods for sparse matrices; the latter focuses on symmetric positive definite matrices. Gilbert [101] and Liu [150] provide an overview of much of the graph theory related to sparse direct methods. Stewart [192] provides a tutorial-level description of sparse Cholesky. Gould, Hu, and Scott [116] survey a wide range of software packages for the factorization of sparse symmetric matrices. Iterative methods for sparse linear systems and the incomplete factorization methods they rely on are discussed by Saad [178], Greenbaum [117], and Barrett et al. [15]. Björck [17] presents direct and iterative methods for sparse least squares problems. Parlett [164] provides an in-depth look at the symmetric eigenvalue problem for sparse matrices. Demmel [39] interleaves a discussion of numerical linear algebra with a description of related software for sparse and dense problems.